Analog Astrogation — Campaign Workbook

2026-08-02

Contents

Phase 0 — Tools

Phase 0.0 — Slide Rule Basics

Type: Tool drill, no mission narrative. **Target time:** ~30 min if the scales are already comfortable; up to an afternoon if not. **Prerequisite:** None. This is the dependency root for everything else in the campaign. **Assumes:** A generic linear slide rule with C, D, A, B, K, S, and T scales (a standard 10” engineering rule — Pickett, K&E, Post, etc. all work). This is the baseline for Phase 0; nothing stops a later stage from calling for a specialized rule (vector rule, log-log duplex for exponentials, a period Trig/Kepler-specific rule, etc.) if the era or problem genuinely warrants it — just note the swap explicitly in that problem’s file when it happens.

A note before you start

You’ve got E6B experience, not linear-rule experience — treat that as a **false friend**, not a head start. The E6B is a *circular* rule built around wind-triangle and fuel-burn conventions (rotating compass rose, specialized scales for TAS/groundspeed). A linear C/D/A/B rule shares the underlying logarithmic-scale principle but none of the E6B’s specialized layout. Two habits from the E6B will actively work against you here:

- **No fixed “index” story.** On the E6B you’re often reading off a wind triangle with a consistent visual frame. On a linear rule, the C and D scales slide relative to each other and there’s no rotating dial to anchor your eye — you have to track which index (1 at the left end of C, or the 10 at the right end) you’re using on a given problem, and that choice changes every time.
- **Decimal placement is entirely on you.** The E6B’s problems are bounded (airspeeds, fuel flows) so wrong-order-of-magnitude answers are obviously wrong at a glance. A slide rule reads the same for 2.35, 23.5, and 235 — the scale only gives you the *significant digits*. You have to track the decimal point yourself with a quick mental estimate before you trust the rule’s answer. This is arguably the single most important habit in this whole drill, and it’s the one place hand computation fails silently if you skip it.

Scale reference

Scale	What it is	Use
C, D	Identical logarithmic scales, 1–10	Multiplication, division
A, B	Two decades (1–100) squeezed into the C/D length	Squares, square roots
K	Three decades (1–1000)	Cubes, cube roots
S	Sine scale, angle in degrees	sin(θ) read against C/D
T	Tangent scale, angle in degrees	tan(θ) read against C/D

Standard technique, if it’s been a while: to multiply $a \times b$, set the left (or right) index of C over a on D, slide the cursor to b on C, read the result on D. Division reverses this. Squares: set cursor on D, read A. Cubes: set cursor on D, read K. Sine/tangent: set cursor to the angle on S (or T), read the corresponding value on C (aligned back to D for a combined calculation).

Drill 1 — Multiplication and division (C/D scale)

Work these on the C/D scale only. Estimate the decimal placement mentally *before* you read the rule, then confirm.

1. 2.35 × 4.12
2. 7.68 × 1.94

3. 3.14×6.02
4. $56.3 \div 8.15$
5. $128 \div 3.7$
6. 9.81×2.71828
7. 0.0456×273
8. $84.2 \div 0.0692$
9. $(4.5 \times 6.3) \div 2.1$ — do this as one continuous setting, without resetting the rule between the multiply and the divide. That chaining is the actual skill; a real hand-computer never re-set the rule for an intermediate result if they could avoid it.

Drill 2 — Squares, cubes, and roots (A/B, K scale)

1. 3.85^2
2. 12.7^2
3. 0.542^2
4. 2.15^3
5. $\sqrt{56.3}$
6. $\sqrt{842}$
7. $\sqrt[3]{47.5}$

Drill 3 — Trig scales (S, T)

1. $\sin(23.5°)$
2. $\sin(67°)$
3. $\tan(15°)$
4. $\tan(52°)$
5. $45.0 \times \sin(30.0°)$ — combined trig + multiply, one setting
6. $68.3 \div \tan(41.0°)$ — combined trig + divide, one setting

---

Worksheet

Fill in your slide rule reading and your decimal-placement estimate *before* checking the answer key.

Problem	Your estimate (order of magnitude)	Your slide rule reading
1.1		
1.2		
1.3		
1.4		
1.5		
1.6		
1.7		
1.8		
1.9		
2.1		
2.2		
2.3		
2.4		

Problem	Your estimate (order of magnitude)	Your slide rule reading
2.5		
2.6		
2.7		
3.1		
3.2		
3.3		
3.4		
3.5		
3.6		

Answer key + tolerance

A well-made 10” rule read carefully gets you roughly ±0.5% on C/D, ±1% on A/B/K (the scale is compressed, so precision is inherently coarser), and ±1–2% on S/T (worse near the ends of the scale — very small angles on S, angles approaching 90° on T). These aren’t arbitrary — they’re the same order of error a real 1950s–60s hand computer lived with, and it’s why period procedures cross-checked results and worked to a *tolerance band* rather than treating any single hand answer as exact.

Drill 1 1. 9.68 (band: 9.63–9.73) 2. 14.9 (14.83–14.97) 3. 18.9 (18.81–18.99) 4. 6.91 (6.87–6.94) 5. 34.6 (34.4–34.8) 6. 26.7 (26.6–26.8) 7. 12.4 (12.3–12.5) 8. 1220 (1210–1230) 9. 13.5 (13.4–13.6)

Drill 2 1. 14.8 (14.7–15.0) 2. 161 (159–163) 3. 0.294 (0.291–0.297) 4. 9.94 (9.84–10.04) 5. 7.50 (7.43–7.58) 6. 29.0 (28.7–29.3) 7. 3.62 (3.58–3.66)

Drill 3 1. 0.399 (0.391–0.407) 2. 0.921 (0.912–0.930) 3. 0.268 (0.263–0.273) 4. 1.28 (1.26–1.30) 5. 22.5 (22.3–22.7) 6. 78.6 (77.0–80.2 — tan is steep near 41–45°, so this one’s naturally looser)

Verification step

Recompute all three drills on a calculator (full precision) and log your percent error against the band above for each problem. Two things to look for, not just “did I land in the band”:

- **Where your error is largest.** If it clusters in Drill 3 or at the small end of Drill 2 (0.542^2), that’s expected — scale compression, not you doing something wrong.
- **Whether your error is systematic.** A consistent one-direction bias (always reading slightly high, say) usually means a cursor-alignment habit to fix, not random reading noise. Worth knowing now, before it’s buried inside a multi-step orbital problem in Stage 1+.

Phase 0.1 — Trig/Log Table Interpolation

Type: Tool drill, no mission narrative. **Target time:** ~30 min if interpolation is already comfortable; longer if not. **Prerequisite:** 0.0 Slide rule basics. **Assumes:** You have a 4- or 5-place log table and a matching trig table (sine/tangent, tenths of a degree). The excerpts below are representative of real period tables in layout and precision — treat them as if torn from an actual reference; a period-correct edition (CRC Standard Mathematical Tables or similar) will be linked in `reference/tables.md` per stage as sourcing is done.

Why this matters

A table only gives you exact values at its tabulated points. Everything between two entries has to be interpolated — and every hand-computation procedure from this era (Gauss's method, Kepler's-equation solving, orbit determination worksheets) leans on this constantly. Get sloppy here and the error propagates into every later step of a multi-stage hand solution.

Method: linear interpolation

Given a table with entries at x_0 and $x_1 = x_0 + \Delta x$, and a target value x between them:

$$f(x) \approx f(x_0) + [(x - x_0) / \Delta x] \cdot [f(x_1) - f(x_0)]$$

In words: find how far across the interval your target sits (as a fraction), then move that same fraction of the way between the two tabulated values. This works well when the function is close to a straight line over the interval — which is the whole design goal of a good table (fine enough spacing that linear interpolation is trustworthy). It breaks down where the function curves sharply within one interval — see Drill 3 below.

Drill 1 — Log table interpolation

Table excerpt: $\log_{10}(x)$ for x = 1.00 to 1.10

x	$\log_{10}(x)$
1.00	0.00000
1.01	0.00432
1.02	0.00860
1.03	0.01284
1.04	0.01703
1.05	0.02119
1.06	0.02531
1.07	0.02938
1.08	0.03342
1.09	0.03743
1.10	0.04139

Interpolate:

1. $\log_{10}(1.034)$
2. $\log_{10}(1.067)$
3. $\log_{10}(1.023)$
4. $\log_{10}(1.089)$

Drill 2 — Trig table interpolation

Table excerpt: sin(θ) for θ = 23.0° to 24.0°

θ	sin(θ)
23.0°	0.39073
23.1°	0.39234
23.2°	0.39394
23.3°	0.39555
23.4°	0.39715
23.5°	0.39875
23.6°	0.40035
23.7°	0.40195
23.8°	0.40354
23.9°	0.40514
24.0°	0.40674

Interpolate:

1. sin(23.34°)
2. sin(23.78°)
3. sin(23.06°)

Drill 3 — When linear interpolation isn't good enough

Table excerpt: tan(θ) near a steep region

θ	tan(θ)
80.0°	5.67128
80.5°	5.97576
81.0°	6.31375

Notice the differences aren't constant (0.30448, then 0.33799) — the function is curving noticeably even over half-degree steps. This is what "linear interpolation isn't accurate enough" looks like in practice.

1. Interpolate tan(80.3°) two ways: (a) using only the 80.0° and 81.0° entries (coarse table, 1° spacing), and (b) using the 80.0° and 80.5° entries (finer table, 0.5° spacing). Record both.
2. Compare the two answers. The gap between them is the real cost of using too coarse a table near a steep region of the function — this is exactly why period trig tables near 80–90° were often tabulated at finer spacing (arcminutes, not tenths of a degree), or came with a second-difference correction column for exactly this situation.

Hint (read only if you're stuck)

Drill 1, Problem 1 — worked breakdown

$\log_{10}(1.034)$ sits between the 1.03 and 1.04 rows.

- Fraction across the interval: (1.034 − 1.03) / (1.04 − 1.03) = 0.4
- Table values: $\log_{10}(1.03) = 0.01284$, $\log_{10}(1.04) = 0.01703$
- Difference: 0.01703 − 0.01284 = 0.00419
- Interpolated value: 0.01284 + 0.4 × 0.00419 = 0.01284 + 0.001676 ≈ **0.01452**

No slide rule needed for this one — the fraction (0.4) is clean enough to do the multiply in your head or with a quick C/D scale check if you want to confirm the 0.4 × 0.00419 step.

Drill 3, Problem 1 — worked breakdown

(a) Coarse (80.0°/81.0°): fraction = 0.3. Difference = 6.31375 − 5.67128 = 0.64247. Interpolated = 5.67128 + 0.3 × 0.64247 ≈ **5.864**.

(b) Fine (80.0°/80.5°): target 80.3° is 0.6 of the way across this half-degree interval. Difference = 5.97576 − 5.67128 = 0.30448. Interpolated = 5.67128 + 0.6 × 0.30448 ≈ **5.854**.

The two methods disagree by about 0.01 (roughly 0.17%) — small in isolation, but exactly the kind of error a real tracking-station computer couldn't afford to compound across a multi-step solution. This is why finer-spaced tables (or correction columns) existed for exactly this region of the tangent function.

Answer key + tolerance

Drill 1 (tolerance: ±0.00003 — log tables interpolate very cleanly) 1. $\log_{10}(1.034) \approx 0.01452$ 2. $\log_{10}(1.067) \approx 0.02816$ 3. $\log_{10}(1.023) \approx 0.00987$ 4. $\log_{10}(1.089) \approx 0.03703$

Drill 2 (tolerance: ±0.0001) 1. sin(23.34°) ≈ 0.39620 2. sin(23.78°) ≈ 0.40482 3. sin(23.06°) ≈ 0.39169

Drill 3 1a. tan(80.3°), coarse table ≈ 5.864 1b. tan(80.3°), fine table ≈ 5.854 (True value ≈ 5.850 — the fine-table answer is noticeably closer.)

Verification step

Recompute Drill 1 and Drill 2 answers on a calculator at full precision and confirm you're within tolerance. For Drill 3, calculate the true value of tan(80.3°) and note which interpolation (coarse or fine) came closer, and by how much — that gap is the number you'd carry forward as your error budget if this table were all you had in the field.

Phase 0.2 — Kepler’s Equation by Hand

Type: Tool drill, no mission narrative. **Target time:** ~20 minutes per example once the iteration pattern clicks — budget longer the first time through. **Prerequisite:** 0.0 Slide rule basics, 0.1 Trig/log table interpolation.

This is the drill the handoff notes call out specifically: solving Kepler’s equation by hand, iteratively, with a slide rule and trig tables, is something nobody in the group has actually done — even people with graduate-level orbital mechanics training almost always learned it as code, not as a hand procedure. This is where that changes.

The equation

Kepler’s equation relates mean anomaly (time-like, uniform) to eccentric anomaly (geometric, on the auxiliary circle):

```
M = E − e·sin(E)          (radians)
```

Worked in **degrees** (the practical choice with a degree-based trig table), this becomes:

```
M = E − k·e·sin(E)          where k = 180/π ≈ 57.29578
```

There’s no closed-form solution for E — it’s solved iteratively. Newton-Raphson on this equation, in degrees:

```
g(E)  = E − k·e·sin(E) − M
g'(E) = 1 − e·cos(E)
E_new = E − g(E)/g'(E)
```

Watch the units carefully here — this is the single most common place to introduce an error. `g'(E) = 1 − e·cos(E)` has **no** factor of k in it, even though `g(E)` does. That’s not a typo: differentiating `sin(E)` with respect to E when E is expressed in degrees pulls in a factor of π/180 from the chain rule, which exactly cancels the k in the sine term. If you find yourself writing `1 − k·e·cos(E)`, stop — that’s the mistake to check for first.

A good starting guess: **E_0 = M**. For higher eccentricity, $E_0 = M + k\cdot e\cdot \sin(M)$ converges faster (it’s literally the first pass of simple successive substitution) — worth trying on Example 2 below if you want to compare iteration counts.

Reference material

Trig table: use the 0.1 excerpt/technique (interpolate sin and cos to at least 5 decimal places — errors here feed directly into g(E) and g’(E)).

Period-authentic note: purpose-built tables existed historically for solving Kepler’s equation directly by lookup rather than iteration (e.g. tables of E as a function of M and e, in the tradition of Bauschinger’s orbit-determination tables). Sourcing and linking a genuine period edition of one of these is flagged as a follow-up for `reference/tables.md` — not yet done, so this drill uses direct Newton-Raphson iteration instead, which is itself a legitimate period-appropriate method, just more laborious than a purpose-built lookup table would have been.

Example 1 — Low eccentricity

Given: e = 0.100, M = 30.000°

Iterate E_new = E − g(E)/g’(E) starting from E_0 = M, until |ΔE| is comfortably below your target precision (aim for < 0.01°).

Example 2 — Higher eccentricity

Given: e = 0.400, M = 45.000°

Same procedure. Expect more iterations and a bigger first correction — this is exactly why higher-eccentricity orbits were the harder case for real human computers, and why a smarter initial guess mattered more as e grew.

Once E converges for either example, get the true anomaly ν:

```
tan(ν/2) = √[(1+e)/(1−e)] · tan(E/2)
```

Worksheet

Use one row per Newton-Raphson iteration. Add rows as needed.

Example 1 (e = 0.100, M = 30.000°)

n	E_n (°)	sin(E_n)	cos(E_n)	g(E_n)	g'(E_n)	ΔE
0	30.000					
1						
2						

Converged E = __________° → ν = __________°

Example 2 (e = 0.400, M = 45.000°)

n	E_n (°)	sin(E_n)	cos(E_n)	g(E_n)	g'(E_n)	ΔE
0	45.000					
1						
2						
3						

Converged E = __________° → ν = __________°

Hint (read only if you're stuck)

Example 1, iteration 0 — worked breakdown

E_0 = 30.000°. From the trig table (or Drill 2 in 0.1): sin(30°) = 0.50000, cos(30°) = 0.86603.

- k·e·sin(E_0) = 57.29578 × 0.100 × 0.50000 = 2.86479 — do this as one chained slide-rule setting: 57.29578 × 0.1 on C/D first (= 5.72958), then × 0.5 without resetting (= 2.86479).
- g(E_0) = 30.000 − 2.86479 − 30.000 = **−2.86479**
- g'(E_0) = 1 − 0.100 × 0.86603 = 1 − 0.08660 = **0.91340**
- ΔE = −g(E_0)/g'(E_0) = −(−2.86479)/0.91340 = **+3.1367°**
- E_1 = 30.000 + 3.1367 = **33.1367°**

Continue the same pattern for iteration 1: look up sin/cos of 33.1367° (interpolating in your trig table), recompute g and g′, and you should see ΔE shrink to well under a tenth of a degree — that's the quadratic convergence Newton-Raphson is known for.

Why Example 2 takes more iterations

The first correction in Example 2 is much bigger (roughly +22.6° on the first step, versus +3.1° in Example 1) because both e and the initial residual are larger. Newton-Raphson still converges — it just needs one or two extra passes to settle below 0.01°. If your second iteration overshoots past the converged value and swings back, that's normal behavior for this equation at moderate eccentricity, not a sign you've made an arithmetic error — check the *trend* (ΔE shrinking each pass) rather than expecting smooth monotonic approach.

Answer key + tolerance

Tolerance reflects realistic hand-computation precision compounding through several trig lookups and a few iterations: ±0.05° on E, ±0.1° on ν (the true-anomaly conversion has its own tangent-half-angle lookup, so it carries a bit more accumulated error).

Example 1 (e = 0.100, M = 30.000°) - Converges in ~2 iterations - E ≈ 33.13° - ν ≈ 36.40°

Example 2 (e = 0.400, M = 45.000°) - Converges in ~3 iterations - E ≈ 65.92° - ν ≈ 89.49°

Notice how much farther ν pulls ahead of E (and E ahead of M) as eccentricity increases — 36.4° vs 33.1° vs 30.0° in Example 1, but 89.5° vs 65.9° vs 45.0° in Example 2. That spread *is* the geometric meaning of eccentric vs. true anomaly, and it's worth sitting with for a second rather than just checking the number.

Verification step

Recompute both examples with a calculator or a quick script carrying full double-precision Newton-Raphson to convergence, and compare against your hand-iterated E and ν. Note two things: how many iterations you needed by hand versus how many the exact computation takes to reach the same precision, and whether your hand answer's error is dominated by trig-table interpolation error (from 0.1) or by stopping the iteration slightly early. Distinguishing those two error sources is exactly the kind of judgment a real tracking-station human computer had to develop.

Phase 0.3 — Triangle-Solving Drills

Type: Tool drill, no mission narrative. **Target time:** ~30 min. **Prerequisite:** 0.0 Slide rule basics, 0.1 Trig/log table interpolation.

Much of hand orbital mechanics — tracking geometry, Gauss's method, range- and-bearing problems — reduces at some point to careful triangle solving. This drill covers the three tools you'll lean on repeatedly: law of cosines, law of sines, and the ambiguous SSA case (which trips up more people than it should, and is worth meeting here rather than mid-problem later).

Law of cosines

$$c^2 = a^2 + b^2 - 2ab\cdot\cos(C) \qquad \text{(SAS: two sides + included angle} \rightarrow \text{third side)}$$
$$\cos(C) = (a^2 + b^2 - c^2) / (2ab) \qquad \text{(SSS: three sides} \rightarrow \text{any angle)}$$

Law of sines

$$a/\sin(A) = b/\sin(B) = c/\sin(C)$$

The ambiguous case (SSA)

Given two sides and a non-included angle, there can be **zero, one, or two** valid triangles. If the given angle is acute and the side opposite it is shorter than the other given side, check both the arcsine solution and its supplement (180° − that angle) — both may produce a valid triangle. This is a genuine geometric ambiguity, not a mistake to avoid; real tracking data resolved it with a second observation or physical reasoning about which solution made sense.

Drill 1 — SAS

Sides a = 42.3, b = 57.8, included angle C = 63.5°. Find side c.

Drill 2 — SSS

Sides a = 35.0, b = 48.0, c = 61.0. Find angle C (opposite the longest side).

Drill 3 — ASA

Angle A = 41.0°, angle B = 67.0°, side a = 28.5 (opposite A). Find angle C, side b, and side c.

Drill 4 — SSA (ambiguous case)

Side a = 22.0, side b = 30.0, angle A = 25.0° (opposite side a). Find all valid solutions for angle B, and complete each triangle (angle C and side c).

Worksheet

Drill	Given	Working	Answer
1	a=42.3, b=57.8, C=63.5°		c =
2	a=35.0, b=48.0, c=61.0		C =
3	A=41.0°, B=67.0°, a=28.5		C=___ b=___ c=___
4a	a=22.0, b=30.0, A=25.0°		B=___ C=___ c=___
4b	(second solution)		B=___ C=___ c=___

Hint (read only if you're stuck)

Drill 1 — worked breakdown

$c^2 = a^2 + b^2 - 2ab \cdot \cos(C)$

- $a^2 = 42.3^2 = 1789.29$ — square on the A/B scale (0.0 Drill 2 skill)
- $b^2 = 57.8^2 = 3340.84$
- Sum: 5130.13
- $2ab = 2 \times 42.3 \times 57.8 = 4889.88$ — chain multiply on C/D without resetting
- $\cos(63.5°) = 0.44620$ — trig table, interpolating between 63° and 64° entries
- $2ab \cdot \cos(C) = 4889.88 \times 0.44620 \approx 2181.9$
- $c^2 = 5130.13 - 2181.9 = 2948.2$
- $c = \sqrt{2948.2} \approx$ **54.3** — square root via A/B scale (reverse of the squaring operation)

Every step here is a tool you already drilled in 0.0 and 0.1 — this problem is really just about chaining them in the right order without losing track of intermediate values. Write down each intermediate result; don't try to carry more than one running value in your head.

Drill 4 — why there are two answers

$\sin(B) = b \cdot \sin(A)/a = 30.0 \times \sin(25.0°) / 22.0 = 30.0 \times 0.42262 / 22.0 \approx 0.5763$

This gives $B \approx 35.2°$ **or** $B \approx 180° - 35.2° = 144.8°$. Check both: does $A + B$ stay under 180° in each case?

- $25.0° + 35.2° = 60.2°$ — valid, leaves 119.8° for C
- $25.0° + 144.8° = 169.8°$ — also valid, leaves only 10.2° for C

Both triangles are geometrically real. This is the case to internalize now: whenever $a > (b \cdot \sin A)$ but $a < b$, expect two solutions, not one — later, when this shows up buried inside a tracking-geometry problem, you want to recognize it immediately rather than quietly picking one root and moving on.

Answer key + tolerance

Tolerance: ±0.3% on lengths, ±0.2° on angles — consistent with chaining a few slide-rule operations and one or two trig-table lookups per problem.

Drill 1: $c \approx 54.3$

Drill 2: $C \approx 93.3°$

Drill 3: $C = 72.0°$ exact $(180 - 41 - 67)$, $b \approx 40.0$, $c \approx 41.3$

Drill 4: - Solution a: $B \approx 35.2°$, $C \approx 119.8°$, $c \approx 45.2$ - Solution b: $B \approx 144.8°$, $C \approx 10.2°$, $c \approx 9.2$

Verification step

Recompute all four drills at calculator precision and confirm you're within tolerance. For Drill 4 specifically, verify both solutions independently using the law of cosines instead of the law of sines ($c^2 = a^2 + b^2 - 2ab \cdot \cos(C)$ with your solved C) — if both cross-checks close, that's good confirmation you found genuinely valid triangles and not an arithmetic coincidence.

Stage 1 — Suborbital Ballistics (Redstone/Mercury-Redstone, ~1961)

Stage 1.0 — Range Table Targeting (Powered Ascent + Ballistic Arc)

Era: Redstone/Mercury-Redstone class, ~1961 (fictional program). **Prerequisite:** Phase 0 (all four drills). **Depends on:** 0.0, 0.1 (slide rule, table interpolation) directly; 0.3 (triangle solving) indirectly, for the same "expect more than one valid answer" habit.

This version deliberately simplifies two things — a constant Isp and a given, un-derived loss budget — to keep the first Stage 1 problem to about half an hour. For the harder, more complete version of this same ascent (where both of those are derived rather than assumed), see **1.1** (liftoff sanity check + gravity-turn loss, first-principles) and **1.2** (non-constant Isp, full afternoon).

Narrative frame

You're the flight dynamics engineer for **Project Arrowhead**, a fictional 1961 suborbital test program flying the **Redwing-3** booster out of a fictional coastal range. Range Safety has assigned tomorrow's flight an impact zone centered **210.0 nautical miles** downrange of the pad, inside a larger safety corridor. Your job: using the vehicle's known performance and a burnout altitude fixed by the guidance program, build the range table for this vehicle and tell the pad crew what burnout flight path angle(s) will put the impact where Range Safety wants it — and confirm there's more than one way to get there.

Given data

Vehicle performance (powered ascent): - Gross liftoff weight, W_0 = 55,000 lbf - Propellant weight burned to cutoff, W_p = 40,000 lbf - Effective specific impulse, I_{sp} = 235 s (treated as constant — real engines vary I_{sp} with altitude, but a single effective value is the period-standard simplification for a first-pass performance estimate; see 1.2 for the non-constant version) - Burn time, t_b = 143 s - Gravity + drag loss allowance, Δv_losses = 3540 ft/s (derived in 1.1 from a first-principles gravity-turn/drag integration — treat it as a given here, the way a real FDO would receive it from the trajectory design group, having not run that study themselves)

Ballistic arc: - Burnout altitude, h_bo = 100,000 ft (fixed by the guidance program, independent of flight path angle — this matches the altitude 1.1's gravity-turn study actually converges to, which is not a coincidence: the pitch program in 1.1 was designed to hit it) - Standard gravity, g = 32.174 ft/s² (treated as constant with altitude — valid at these altitudes to within a few percent, and consistent with the flat, non-rotating Earth model used throughout this problem) - Assume impact at sea level (flat, non-rotating Earth — no curvature correction at this range; that correction is flagged as a natural extension for a later, longer-range stage)

Target: impact range = 210.0 nautical miles from the pad (1 nm = 6076.12 ft)

Authentic method + reference material

Powered ascent — Tsiolkovsky rocket equation (Tsiolkovsky, 1903; in active use by every rocket engineer of this era):

```
Δv_ideal = c · ln(W₀/W₁)          c = Iₛₚ · g₀
V_bo = Δv_ideal − Δv_losses
```

where $W_1 = W_0 - W_p$ is burnout weight.

Ballistic arc — flat-Earth, uniform-gravity vacuum trajectory. This is the same mathematics behind WWII- and V2-era artillery/rocket range tables, which the Redstone program inherited directly through von Braun's team. It is *not* the full spherical-Earth treatment (Bate, Mueller & White, Ch. 4, "Ballistic Missile Trajectories," which treats the free-flight arc as a conic with one focus at Earth's center) — that's intentionally saved for a later, longer-range stage, since Stage 1 is scoped to stay out of orbital mechanics. At suborbital ranges like this one, the flat-Earth form is an accurate and period-appropriate simplification, not a shortcut that sacrifices authenticity.

With burnout velocity V_bo at flight path angle γ (measured above local horizontal) and altitude h_bo:

```
v_y0 = V_bo · sin(γ)        v_x0 = V_bo · cos(γ)

t_impact = [v_y0 + √(v_y0² + 2·g·h_bo)] / g
```

```
R = v_x0 · t_impact

h_apex = h_bo + v_y0² / (2g)
```

The graphical range-table method: rather than solving R(γ) = 210.0 nm directly (it's not algebraically clean to invert), compute R for a spread of γ values, sketch R vs. γ by hand, and read the answer(s) off the curve. This is exactly how real range tables were built and used operationally — a chart, not an equation to invert on demand. Expect the curve to be non-monotonic near its peak, which means **two** valid flight path angles can produce the same range: a flatter, faster-arriving trajectory and a more lofted one. Same ambiguity in spirit as the SSA triangle case in 0.3 — don't be surprised by it, and don't discard one root as "wrong."

Worksheet

Part A — Burnout velocity

Quantity	Value
$W_1 = W_0 - W_p$	
$c = I_{sp} \cdot g_0$	
W_0/W_1	
$\ln(W_0/W_1)$	
$\Delta v\_ideal = c \cdot \ln(W_0/W_1)$	
$V\_bo = \Delta v\_ideal - \Delta v\_losses$	

Part B — Range table. Fill in for each γ. Carry v_y0^2 through in full (you'll reuse it for the apex altitude too — don't recompute).

γ	sin γ	cos γ	v_y0	v_x0	v_y0^2	t_impact (s)	R (ft)	R (nm)	h_apex (ft)
30°									
35°									
40°									
45°									
50°									
55°									

Part C — Graphical solution. Plot R (nm, vertical axis) vs. γ (horizontal axis, 30°–55°) on the grid below using your six points from Part B. Draw a horizontal line at R = 210.0 nm and read off both crossings.

Low-angle solution: γ = __________° High-angle solution: γ = __________°

Hint (read only if you're stuck)

Part A — worked breakdown

- W_1 = 55,000 − 40,000 = **15,000 lbf**
- c = 235 × 32.174 = **7560.9 ft/s**

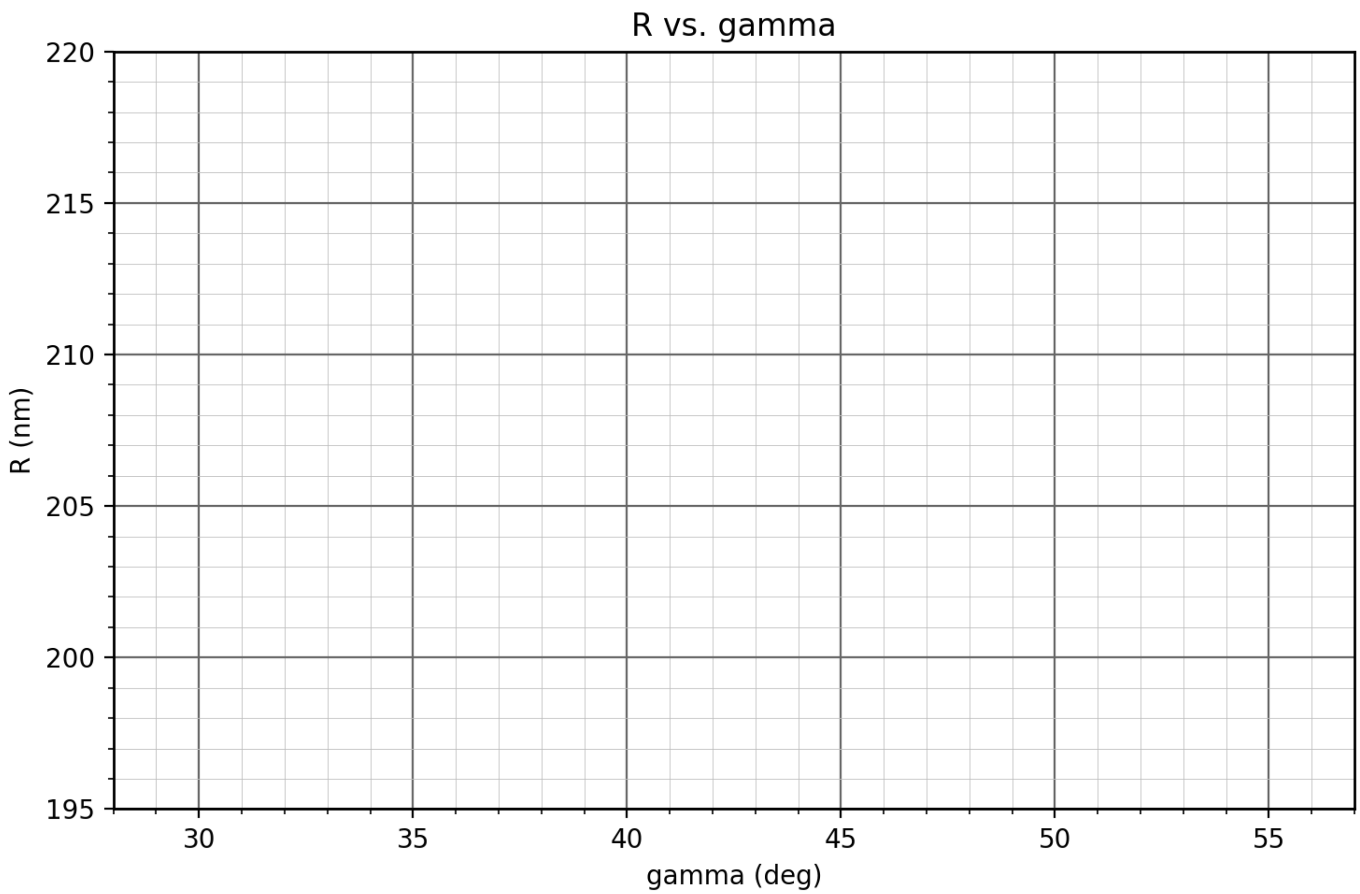

Figure 1: R vs. gamma

- W_0/W_1 = 55,000/15,000 = **3.6667**
- ln(3.6667) ≈ **1.2993** — via $\log_{10}$ on the L scale (or table) × 2.302585: $\log_{10}$(3.6667) ≈ 0.56425, × 2.302585 ≈ 1.2993
- Δv_ideal = 7560.9 × 1.2993 ≈ **9824 ft/s**
- V_bo = 9824 − 3540 = **6284 ft/s**

Part B, γ = 45° row — worked breakdown

- sin 45° = cos 45° = 0.70711
- v_y0 = v_x0 = 6284 × 0.70711 ≈ **4443 ft/s**
- v_y0^2 ≈ **19,750,000** ft^2/s^2
- 2·g·h_bo = 2 × 32.174 × 100,000 = **6,434,800** ft^2/s^2
- v_y0^2 + 2·g·h_bo ≈ 26,184,800 → √ ≈ **5117** ft/s
- t_impact = (4443 + 5117)/32.174 ≈ **297.2 s**
- R = 4443 × 297.2 ≈ 1,320,300 ft → **≈ 217.3 nm**
- h_apex = 100,000 + 19,750,000/64.348 ≈ **406,900 ft**

Chain the multiply/divide/sqrt steps on the slide rule without resetting where you can — this row has five slide-rule operations back to back, which is exactly the kind of chaining 0.0 Drill 1, Problem 9 was building toward.

Answer key + tolerance

Part A: V_bo ≈ 6284 ft/s (tolerance: ±60 ft/s, ~1%)

Part B (tolerance: ±0.5% on R, ±1% on h_apex)

γ	R (nm)	h_apex (ft)
30°	199.9	253,400
35°	211.0	301,900
40°	216.9	353,600
45°	217.3	406,900
50°	211.9	460,100
55°	200.7	511,800

The curve peaks between 40° and 45° rather than exactly at 45° — that's the real effect of burnout altitude being nonzero (launching from 100,000 ft down to a sea-level impact isn't quite symmetric with the textbook flat-ground case). Small effect here since h_bo is small compared to R, but it's genuine, not noise.

Part C (tolerance: ±1.5°, consistent with reading a hand-drawn chart) - Low-angle solution: γ ≈ **34.5°** - High-angle solution: γ ≈ **51.1°**

Both are legitimate answers to "what flight path angle hits 210 nm" — tell the pad crew there's a choice to make (faster/flatter vs. more lofted, with a correspondingly lower or higher apex), not that there's one right number.

Verification step

Recompute Part B in STK or your Astrogator clone as a simple flat, non- rotating-Earth vacuum trajectory from the same burnout state, and confirm your six hand-computed ranges land within tolerance. Then find both γ solutions for R = 210.0 nm numerically (bisection or Newton's method against R(γ) − 210.0 = 0) and compare against your graphically-read values — that gap is your graphical-reading error budget, separate from your arithmetic error budget from Part B.

As a bonus check worth doing once: rerun the 45° case in STK using a spherical, rotating Earth instead of the flat-Earth model used here, and see how far the actual impact point shifts. At ~200 nm this should be a modest correction — worth having the number in hand before a later stage pushes range far enough that the flat-Earth assumption stops being defensible.

Stage 1.1 — Liftoff Check & Gravity-Turn Loss (First Principles)

Era: Redstone/Mercury-Redstone class, ~1961 (fictional program), continuing Project Arrowhead. **Target time:** ~1.5 hours. **Prerequisite:** Stage 1.0 (Range Table Targeting).

Why this problem exists

1.0 handed you Δv_losses as a given number, the way a real FDO would receive it from the trajectory design group. This problem *is* that group's work: where the number actually comes from, and — as a bonus you didn't ask for — a check on whether the vehicle spec you were handed even makes sense.

Part 1 — Sanity-check the vehicle

Before modeling anything about the ascent, check the most basic thing: can this vehicle actually leave the pad? A rocket needs thrust greater than its own weight at ignition, full stop.

Candidate spec (proposed by the vehicle design group): W_0 = 66,000 lbf, W_p = 40,000 lbf burned over t_b = 143 s, I_{sp} = 235 s.

```
ẇ = Wₚ / t_b                  (weight flow rate)
T = ẇ · Iₛₚ                   (thrust, from the definition of Iₛₚ)
T/W₀ = liftoff thrust-to-weight ratio
```

Compute T/W_0 for the original spec. A real booster of this class typically lifts off around $T/W_0 \approx 1.2$; anything at or below 1.0 cannot lift off at all.

What you should find: $T/W_0 \approx 0.996$. This vehicle cannot fly. It was never caught because 1.0's calculation only used the *integrated* rocket equation (total Δv from the mass ratio), which doesn't care about instant-by-instant thrust — a completely reasonable thing to miss if you never build a real time-history of the flight, which is exactly what 1.0 didn't do and this problem does.

Part 2 — Corrected data, and the real ascent

The propulsion group reviewed the spec and traces the error to an overstated liftoff weight. **Corrected value: W_0 = 55,000 lbf** (W_p, t_b, and I_{sp} unchanged). Recompute T/W_0 and confirm it's now reasonable (target: ~1.2).

With the corrected vehicle, the guidance group ran a preliminary trajectory-shaping study — the kind of repetitive, closely-spaced numerical integration that, even in 1961, nobody did by hand end-to-end. That's the "very limited computer" moment for this problem: below is a coarse extract from that run (7 points instead of the hundreds the real integration would use), given to you the way a trajectory analyst would actually receive it.

Programmed pitch maneuver: the guidance program holds the vehicle vertical (γ = 90°) until t_p = 13.0 s, then flies a prescribed pitch profile that eases toward 15° by burnout. (The exact formula isn't needed for this part — it's given in the Verification step below, when you build your own integrator.)

Given — trajectory checkpoints (from the preliminary run):

t (s)	V (ft/s)	γ (°)	h (ft)
0	0.0	90.0	0
13	98.7	90.0	604
39	499.8	46.53	6,485
65	1,282.9	28.25	19,230
91	2,372.1	20.57	37,878
117	3,888.0	17.34	63,427
143	6,284.2	15.98	100,235

Given — vehicle aerodynamic data (from wind tunnel testing): effective drag parameter Cd·A = 8.0 ft^2.

Given — atmosphere model: isothermal exponential atmosphere (the same simplified model Bate, Mueller & White use for atmospheric-drag perturbation work) —

```
ρ(h) = ρ₀ · exp(−h/H)          ρ₀ = 0.0023769 slug/ft³, H = 23,800 ft
```

Authentic method + reference material

Drag force: $D = \tfrac{1}{2}\cdot\rho(h)\cdot V^2\cdot(Cd\cdot A)$ — standard aerodynamic drag equation.

Loss-rate bookkeeping. At any instant, gravity is costing you $g_0\cdot\sin(\gamma)$ of your rocket's Δv-generating capability, and drag is costing you $(D/W)\cdot g_0$. This is genuine physics, not an artifact of this drill — it comes directly from the same equation of motion ($dV/dt = c\cdot\dot{w}/W - D\cdot g_0/W - g_0\sin\gamma$) that a full trajectory integration solves.

The hand technique: compute both loss rates at each of the seven given checkpoints, then integrate (trapezoidal — average the rate at each pair of adjacent checkpoints, multiply by the time between them, sum) to get total gravity loss and total drag loss over the burn. This is *not* the same as integrating the full coupled trajectory yourself — that's a much harder, numerically unstable problem by hand (ask your instructor about the war story where the first attempt at this diverged spectacularly with only five hand-sized steps). Using given trajectory checkpoints and just integrating the loss *rates* sidesteps that entirely, and — surprisingly — still gets you a very good answer, as you'll see in Verification.

Worksheet

Part 1 — original spec:

Quantity	Value
$\dot{w} = W_p/t\_b$	
$T = \dot{w}\cdot I_{sp}$	
T/W_0 (original, 66,000 lbf)	

Part 2a — corrected spec:

Quantity	Value
T/W_0 (corrected, 55,000 lbf)	

Part 2b — loss-rate table. Fill in for each checkpoint.

t (s)	V	h	γ	W(t)	ρ(h)	D	grav. rate = $g_0\sin\gamma$	drag rate = $(D/W)g_0$
0	0.0	0	90.0°					
13	98.7	604	90.0°					
39	499.8	6,485	46.53°					
65	1,282.9	19,230	28.25°					
91	2,372.1	37,878	20.57°					
117	3,888.0	63,421	17.34°					

t (s)	V	h	γ	W(t)	ρ(h)	D	grav. rate = $g_0 \sin\gamma$	drag rate = $(D/W) g_0$
143	6,284.2	100,231	15.98°					

Part 2c — trapezoidal integration.

Interval	Δt (s)	Avg grav. rate × Δt	Avg drag rate × Δt
0–13	13		
13–39	26		
39–65	26		
65–91	26		
91–117	26		
117–143	26		

Total gravity loss = __________ ft/s Total drag loss = __________ ft/s Total Δv_losses = __________ ft/s

Using Δv_ideal = 9824 ft/s (from 1.0's Part A, same vehicle): V_bo = __________ ft/s

---

Hint (read only if you're stuck)

Part 1 — worked breakdown

- $\dot{w}$ = 40,000/143 = **279.72 lbf/s**
- T = 279.72 × 235 = **65,734 lbf**
- T/W_0 = 65,734/66,000 = **0.996**

Under 1.0 — this vehicle physically cannot leave the pad.

Part 2b, t = 39 s row — worked breakdown

- W(39) = 55,000 − 279.72×39 = **44,091 lbf**
- ρ(6,485) = 0.0023769 × exp(−6485/23800) = 0.0023769 × 0.7546 ≈ **0.001794 slug/ft³**
- D = 0.5 × 0.001794 × 499.8² × 8.0 ≈ **1,791 lbf** (small rounding vs. the reference value below — normal for a hand-carried exponential)
- grav rate = 32.174 × sin(46.53°) = 32.174 × 0.7254 ≈ **23.34**
- drag rate = (1,791/44,091) × 32.174 ≈ **1.31**

Answer key + tolerance

Part 1: T/W_0 ≈ **0.996** — vehicle cannot lift off.

Part 2a: T/W_0 ≈ **1.195** — good, matches a typical period booster.

Part 2b/2c (tolerance: ±3% per row — the exponential atmosphere term is the least forgiving thing to carry by hand here):

Interval	Grav. contribution (ft/s)	Drag contribution (ft/s)
0–13	418.3	0.4
13–39	721.8	17.9

Interval	Grav. contribution (ft/s)	Drag contribution (ft/s)
39–65	501.5	96.4
65–91	345.0	233.4
91–117	271.6	342.1
117–143	239.9	343.0

Total gravity loss ≈ 2498 ft/s Total drag loss ≈ 1033 ft/s Total Δv_losses ≈ 3531 ft/s

V_bo ≈ 9824 − 3531 ≈ 6293 ft/s

That's a genuinely large number in absolute terms: this vehicle spends over a third of its ideal Δv capability fighting gravity and drag, which is realistic for a steep suborbital profile that has to reach 100,000 ft.

Verification step

This is the interesting one. Build your own numerical integrator — a short Python script is fine (this is the "modern FORTRAN deck" for this exercise; what matters is the technique, not the language) — using the full coupled equations of motion rather than the given checkpoints:

```
dV/dt = c·ẇ/W(t) − [D(V,h)/W(t)]·g₀ − g₀·sin(γ(t))
dh/dt = V·sin(γ(t))

γ(t) = 90°                              for t < 13
γ(t) = 15° + 75°·exp[−(t−13)/30]        for t ≥ 13
```

with the same W(t), D(V,h), and atmosphere model as above. Integrate from t=0 to t=143 with a fine time step (RK4 recommended, but even a fine-step Euler will converge). You should get approximately:

- **V_bo ≈ 6284 ft/s**
- **h_bo ≈ 100,235 ft**
- **Implied total loss ≈ 3540 ft/s**

Compare three numbers against each other: your hand/checkpoint estimate (≈6293 ft/s), your own fine-integration run, and the reference values above. The checkpoint method should land within about 0.2% of the fine integration — which is a genuinely useful result: it means a handful of well-chosen checkpoints from an expensive computer run let a human do fast, accurate follow-on work (loss budgets, sensitivity checks) without re-running the whole integration by hand. That division of labor — computer for the fine trajectory, hand/slide-rule for everything built on top of it — is close to exactly how this worked in practice by the time Redwing-3's real counterparts were flying.

Stage 1.2 — Non-Constant Isp Ascent (Hard Mode)

Era: Redstone/Mercury-Redstone class, ~1961 (fictional program), continuing Project Arrowhead. **Target time:** a full afternoon. **Prerequisite:** Stage 1.0 (Range Table Targeting), Stage 1.1 (Liftoff Check & Gravity-Turn Loss).

Why this problem exists

1.0 treated specific impulse as a single constant, 235 s, for the whole burn. Real engines don't work that way: thrust is $F = \dot{m} \cdot Ve + (Pe - Pamb) \cdot Ae$ — as the vehicle climbs and ambient pressure Pamb drops, the same engine produces more thrust and a higher effective Isp, right up to a vacuum ceiling. This is standard, period-authentic propulsion analysis (see Sutton, *Rocket Propulsion Elements* — in print and in active engineering use well before 1961) and it is **not** a small effect: most of the pressure drop happens in the first ~100,000–150,000 ft, which is most of this vehicle's burn. Treating Isp as constant, as 1.0 did, understates burnout velocity by a meaningful margin — you'll see by how much below.

This is genuinely harder than 1.0, for a specific reason: Isp now depends on altitude, altitude depends on the velocity history, and velocity depends on Isp. That circular dependency is why this can't be solved in one closed-form step — it has to be marched forward in time, a handful of steps at a time, the same way early trajectory integrators (Gauss-Jackson and similar — already flagged in the campaign's reference list as a harder future problem in its own right) worked before continuous integration was practical by hand. What you're doing here is a simplified version of that same idea.

Given data

Same (corrected) vehicle as 1.0/1.1: W_0 = 55,000 lbf, W_p = 40,000 lbf burned uniformly over t_b = 143 s (weight decreases linearly with time). Standard gravity g_0 = 32.174 ft/s².

Loss budget: Δv_losses = 3540 ft/s, from Stage 1.1's first-principles gravity-turn/drag study.

Engine performance — Isp vs. altitude (from separate propulsion analysis; reflects the nozzle's altitude compensation as ambient pressure drops):

h (ft)	Isp (s)
0	235
20,000	246
40,000	253
60,000	258
80,000	261
100,000	263
120,000	264
≥ 140,000	265 (vacuum plateau)

Ascent altitude profile: reuse the same trajectory checkpoints from Stage 1.1 (same vehicle, same preliminary shaping run) rather than a separately-invented profile:

t (s)	h (ft)
0	0
13	604
39	6,485

t (s)	h (ft)
65	19,230
91	37,878
117	63,427
143	100,235

Explicit simplification, stated up front: this problem does *not* solve for the trajectory shape — the altitude profile above is a given input (1.1's result), not something you derive from the velocity you're computing here. That keeps the coupling one-directional (altitude → Isp → Δv) and hand-tractable. The full version, where Isp-vs-altitude feeds back into the trajectory shape itself, is real and harder — a natural candidate for a future stage, not this one.

Authentic method + reference material

Divide the burn into the six intervals defined by the checkpoints above (note: unequal step sizes — the first is 13 s, the rest are 26 s each, since they come from 1.1's checkpoint times, not an even split). For each step:

```
1. Look up altitude at the START of the step (from the given profile).
2. Interpolate Isp at that altitude.
3. c = Isp · g₀
4. Δv_step = c · ln(W_start/W_end)     [W_end from the linear weight burn]
5. Accumulate: V_cumulative += Δv_step
```

Note the numerical choice being made in step 1: using the altitude at the *start* of each step to look up Isp for that whole step is an explicit (forward-lagged) scheme — it's simple and hand-tractable, but it means each step's Isp is slightly stale (the vehicle is actually higher, and Isp actually slightly better, by the step's end). A more accurate scheme would use the midpoint or average altitude of the step. This is the same kind of approximation-choice tradeoff numerical integrators always face — flagged here so it's a visible decision, not a hidden one.

Worksheet

Step	t_start–t_end (s)	h_start (ft)	Isp (interp.)	c = Isp·g_0	W_start	W_end	ln(W_start/W_end)	Δv_step (ft/s)	V_cumulative (ft/s)
1	0–13	0			55,000	51,364			
2	13–39	604			51,364	44,091			
3	39–65	6,485			44,091	36,818			
4	65–91	19,230			36,818	29,546			
5	91–117	37,878			29,546	22,273			
6	117–143	63,427			22,273	15,000			

Total ideal Δv (altitude-varying Isp) = __________ ft/s

V_bo = Total ideal Δv − 3540 ft/s = __________ ft/s

Compare to 1.0's constant-Isp result (V_bo ≈ 6284 ft/s): difference = __________ ft/s (______ %)

Hint (read only if you're stuck)

Step 1 — worked breakdown

- h_start = 0 → Isp = 235 s (table row, no interpolation needed)
- c = 235 × 32.174 = 7560.9 ft/s
- ln(55,000/51,364) = ln(1.0708) ≈ 0.0684
- Δv_step = 7560.9 × 0.0684 ≈ **517 ft/s**
- V_cumulative = 517 ft/s

Step 4 — worked breakdown (an interpolated Isp row)

- h_start = 19,230 ft, between table rows 0 (Isp=235) and 20,000 (Isp=246)
- Fraction = 19,230/20,000 = 0.9615
- Isp = 235 + 0.9615×(246−235) = 235 + 10.58 = **245.58 s**
- c = 245.58 × 32.174 ≈ 7901.2 ft/s
- ln(36,818/29,546) = ln(1.2461) ≈ 0.2201
- Δv_step ≈ 7901.2 × 0.2201 ≈ **1739 ft/s**
- V_cumulative = 3056.8 (through step 3) + 1739 ≈ **4796 ft/s**

Answer key + tolerance

Tolerance: ±1.5% on the total (six chained slide-rule/log/interpolation sequences compound more error than 1.0's single calculation).

Step	h_start (ft)	Isp (s)	Δv_step (ft/s)	V_cumulative (ft/s)
1	0	235.00	517	517
2	604	235.33	1156	1673
3	6,485	238.57	1384	3057
4	19,230	245.58	1739	4796
5	37,878	252.26	2293	7089
6	63,427	258.51	3288	10,377

Total ideal Δv ≈ 10,377 ft/s (versus 9824 ft/s at constant 235 s Isp in 1.0 — about 5.6% more; less dramatic than a first guess might suggest, because this vehicle spends its first ~65 s still fairly low and slow, where Isp hasn't climbed much yet — most of the Isp benefit shows up in the last two steps, which is also where most of the propellant burns).

V_bo ≈ 10,377 − 3540 ≈ 6837 ft/s (versus 6284 ft/s in 1.0 — about **553 ft/s, or ~8.8% higher**).

That's still not a rounding-level correction. (Optional extension, not required for this problem: redo 1.0's Part B with this V_bo and see how much farther the impact point moves for each flight path angle. Rough expectation, since range scales close to V_bo^2 at fixed γ and h_bo: on the order of 15–18% more range.)

Verification step

Recompute the six-step integration with a calculator at full precision and confirm your hand total is within tolerance. Then try the alternate numerical scheme flagged above: instead of using each step's *starting* altitude to look up Isp, use the *midpoint* altitude (average of the step's start and end altitude) and recompute. Compare the two totals — the gap between them is your estimate of the error introduced by the explicit/lagged scheme, separate from ordinary arithmetic and interpolation error.

If you built the numerical integrator from 1.1's verification step, this is also a natural place to extend it: add the same Isp(h) lookup into that integrator (replacing the constant c with $c(h) = Isp(h) \cdot g_0$) and get a fully self-consistent, altitude-varying-Isp *and* altitude-varying-loss burnout state in one run, rather than the two separate hand studies here and in 1.1. That combined version is the real “what would the computer actually have produced” answer — worth comparing against both hand results once you have it.